



RIDEFLOW

A Comprehensive Ride Hailing Platform

Database Systems Lab Project

AI & DS - Spring 2026

Group Members

Mohid Abbas	Ubaid ur Rehman
24I-0074	24I-0022

Department of Artificial Intelligence & Data Science
FAST National University of Computer and
Emerging Sciences, Islamabad

May 2026

Contents

1	Introduction	5
1.1	Project Overview	5
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Scope of the Project	6
2	System Design	7
2.1	Entity-Relationship Diagram	7
2.2	Database Schema Design	7
2.2.1	Core Tables	7
2.2.2	Relational Schema	8
3	Database Implementation	10
3.1	Basic SQL Queries	10
3.1.1	Query Examples	10
3.2	Aggregate Functions and HAVING Clause	10
3.2.1	Revenue Calculation	10
3.2.2	Driver Performance Analysis	11
3.3	Joins for Complex Reports	11
3.3.1	Multi-Table Trip Report	11
3.4	Views Implementation	11
3.4.1	Active Rides View	11
3.5	Indexes for Performance	12
3.6	Stored Procedures	12
3.6.1	Fare Calculation Procedure	12
3.7	Triggers	13
3.7.1	Payment Status Trigger	13
3.8	Events	13
3.8.1	Promo Code Expiry Event	13
3.9	Data Control Language (DCL)	14
3.9.1	Role-Based Access Control	14
4	Application Development	15
4.1	Technology Stack	15
4.2	Backend Architecture	15
4.2.1	Server.js Structure	15
4.2.2	API Endpoints	16
4.3	Frontend Implementation	16
4.3.1	Responsive Design	16

4.3.2	Rider Dashboard	17
4.3.3	Book with Maps Feature	18
4.3.4	Driver Dashboard	19
4.3.5	Admin Panel	20
5	Features Demonstration	22
5.1	Core Features	22
5.1.1	Authentication System	22
5.1.2	Rider Features	22
5.1.3	Driver Features	22
5.1.4	Admin Features	23
5.2	Advanced Database Features in Action	23
5.2.1	Stored Procedure: Fare Calculation	23
5.2.2	Trigger: Auto-Complete Ride	23
5.2.3	Event: Promo Code Cleanup	23
5.2.4	View: Active Rides Monitoring	24
6	Work Division	25
6.1	Group Member Responsibilities	25
6.2	Contribution Percentage	25
7	Testing and Validation	26
7.1	Testing Strategy	26
7.1.1	Database Testing	26
7.1.2	Application Testing	26
7.2	Test Cases	27
8	Deployment	28
8.1	Local Deployment	28
8.1.1	Prerequisites	28
8.1.2	Installation Steps	28
8.2	Cloud Deployment Options	28
8.2.1	Render.com Deployment	28
8.2.2	Database Hosting	29
9	Conclusion and Future Work	30
9.1	Project Summary	30
9.2	Challenges Faced	30
9.3	Future Enhancements	30
9.4	Learning Outcomes	31
	References	32
A	Complete File Structure	33
B	SQL File Contents Summary	34
B.1	rideflow_relational_schema.sql	34
B.2	rideflow_deliverable3_advanced_sql.sql	34
C	Screenshots Index	35

List of Figures

2.1	Entity-Relationship Diagram for RideFlow Database	7
2.2	Relational Schema with All Tables and Relationships	8
4.1	Login Page - Entry point for all users	16
4.2	Rider Dashboard - Main interface for ride booking	17
4.3	Google Maps Integration - Interactive ride booking	18
4.4	Driver Dashboard - Ride acceptance and earnings	19
4.5	Admin Dashboard - System overview and analytics	20
4.6	User Management - Admin control panel	21
5.1	Dynamic fare calculation with surge pricing	23
5.2	Real-time active rides monitoring using ActiveRidesView	24
6.1	Equal Contribution Distribution	25

List of Tables

2.1	Database Schema Overview	8
4.1	Technology Stack	15
4.2	REST API Endpoints	16
6.1	Work Division Between Group Members	25
7.1	Sample Test Cases	27
C.1	Required Screenshots for Report	35

Chapter 1

Introduction

1.1 Project Overview

RideFlow is a comprehensive ride-hailing platform designed as an academic project for the Database Systems Lab course. The platform implements a complete ride-booking system with three distinct user roles: Riders, Drivers, and Administrators. Built using modern web technologies and MySQL database, RideFlow demonstrates advanced database concepts including SQL queries, joins, views, stored procedures, triggers, events, and role-based access control.

1.2 Problem Statement

Traditional transportation systems often lack real-time tracking, transparent pricing, and efficient matching between riders and drivers. RideFlow addresses these challenges by providing:

- Real-time ride booking and tracking
- Dynamic fare calculation based on distance and time
- Automated driver-rider matching
- Secure payment processing
- Comprehensive admin monitoring and reporting

1.3 Objectives

1. Design and implement a relational database schema for a ride-hailing platform
2. Develop a functional web application with role-based access control
3. Implement advanced SQL features (views, procedures, triggers, events)
4. Create an intuitive user interface for all user roles
5. Demonstrate database security through DCL implementation

1.4 Scope of the Project

The RideFlow platform includes:

- User authentication with role-based access
- Complete rider functionality (booking, history, wallet)
- Driver management (availability, ride acceptance, earnings)
- Administrative control (user management, fare rules, reports)
- Google Maps integration for location services
- Automated business logic through database triggers and events

Chapter 2

System Design

2.1 Entity-Relationship Diagram

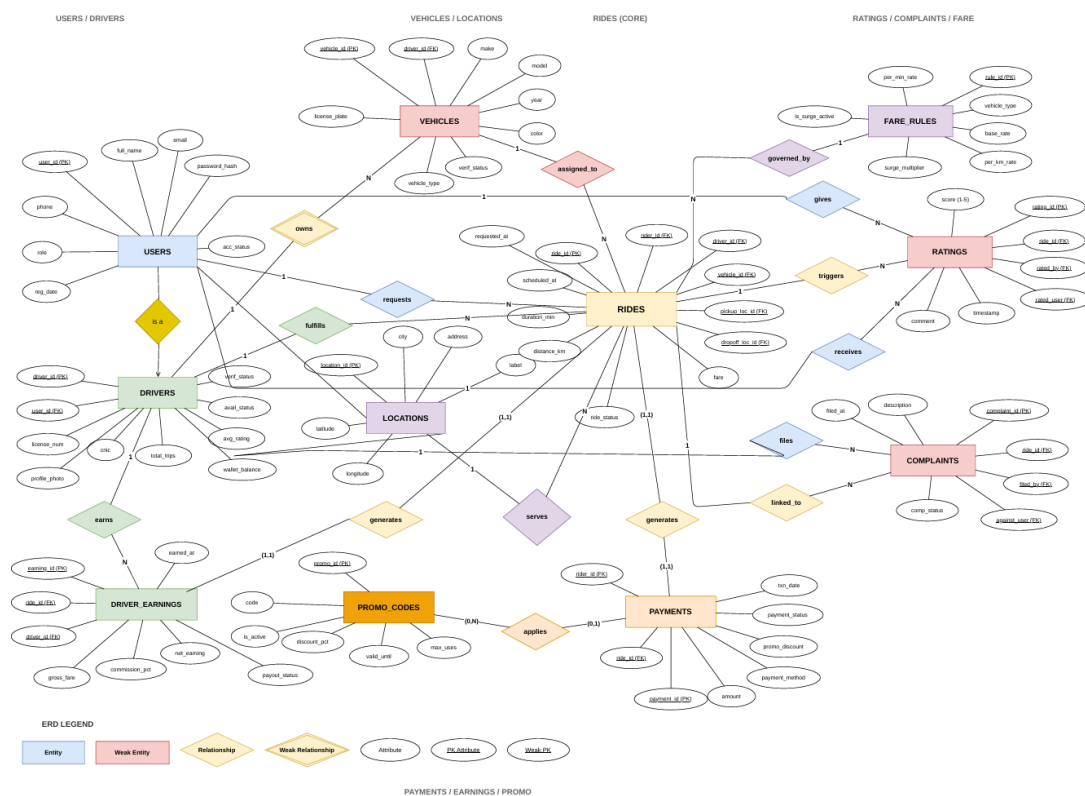


Figure 2.1: Entity-Relationship Diagram for RideFlow Database

The ER diagram in Figure 2.1 illustrates the relationships between all entities in the RideFlow system. Key entities include Users, Riders, Drivers, Rides, Vehicles, Payments, and Locations.

2.2 Database Schema Design

2.2.1 Core Tables

Table 2.1: Database Schema Overview

Table Name	Primary Key	Description
users	user_id	Stores all user credentials and basic info
riders	rider_id	Extended profile for riders
drivers	driver_id	Driver-specific information including license
vehicles	vehicle_id	Vehicle details linked to drivers
rides	ride_id	Core ride booking information
locations	location_id	Geographic location data
payments	payment_id	Payment transaction records
ratings	rating_id	Driver and ride ratings
fare_rules	rule_id	Dynamic pricing configuration
promo_codes	promo_id	Promotional code management

2.2.2 Relational Schema

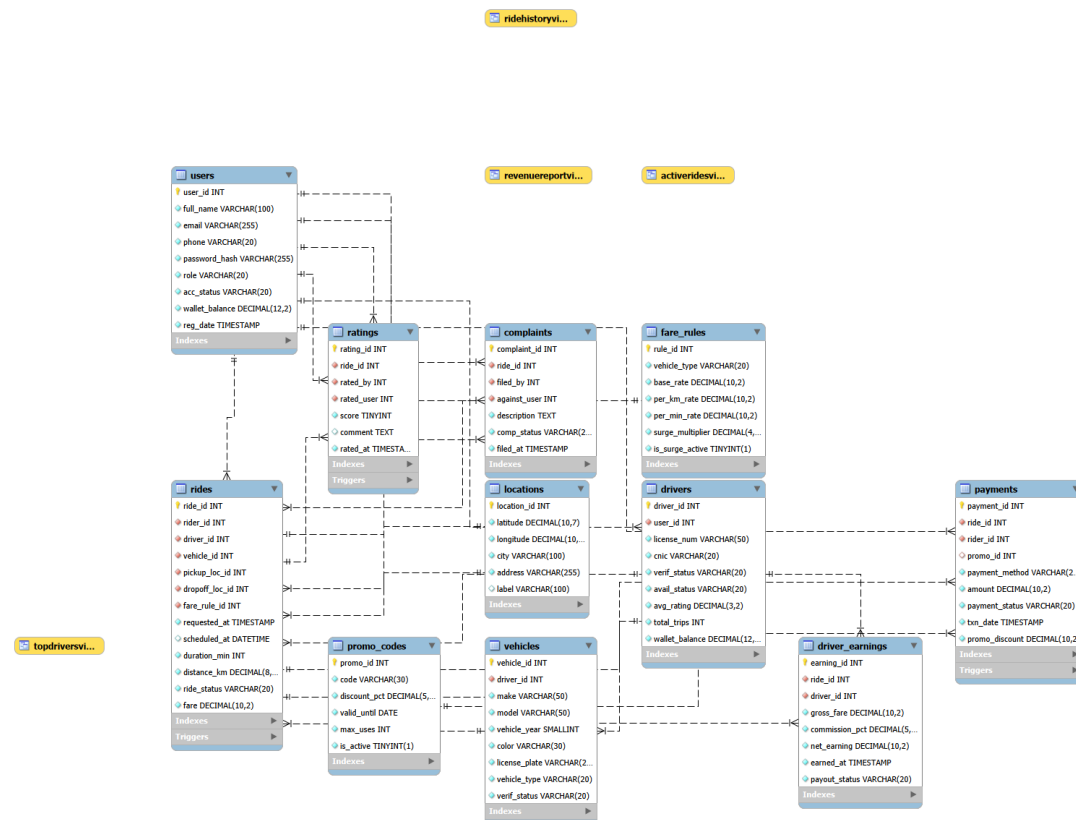


Figure 2.2: Relational Schema with All Tables and Relationships

The system follows a three-tier architecture:

1. **Presentation Layer:** HTML/CSS/JavaScript frontend
2. **Application Layer:** Node.js/Express.js backend

3. **Data Layer:** MySQL database with advanced SQL features

Chapter 3

Database Implementation

3.1 Basic SQL Queries

3.1.1 Query Examples

Listing 3.1: List all completed rides for a specific rider

```
1 SELECT r.ride_id, r.requested_at, r.status,
2       r.pickup_address, r.dropoff_address, r.fare_amount
3 FROM rides r
4 WHERE r.rider_id = 1
5       AND r.status = 'Completed'
6 ORDER BY r.requested_at DESC;
```

Listing 3.2: List all drivers in a city ordered by rating

```
1 SELECT d.driver_id, u.full_name, d.avg_rating,
2       d.total_trips, l.city
3 FROM drivers d
4 JOIN users u ON d.user_id = u.user_id
5 JOIN locations l ON d.current_loc_id = l.location_id
6 WHERE l.city = 'Islamabad'
7 ORDER BY d.avg_rating DESC;
```

3.2 Aggregate Functions and HAVING Clause

3.2.1 Revenue Calculation

Listing 3.3: Calculate total revenue per city

```
1 SELECT l.city,
2       SUM(r.fare_amount) as total_revenue,
3       COUNT(r.ride_id) as total_rides,
4       AVG(r.fare_amount) as avg_fare
5 FROM rides r
6 JOIN locations l ON r.dropoff_loc_id = l.location_id
7 WHERE r.status = 'Completed'
```

```

8 GROUP BY l.city
9 HAVING total_revenue > 5000
10 ORDER BY total_revenue DESC;

```

3.2.2 Driver Performance Analysis

Listing 3.4: Drivers with low ratings requiring attention

```

1 SELECT d.driver_id, u.full_name,
2        AVG(rat.score) as avg_rating,
3        COUNT(DISTINCT ri.ride_id) as total_rides
4 FROM drivers d
5 JOIN users u ON d.user_id = u.user_id
6 LEFT JOIN rides ri ON d.driver_id = ri.driver_id
7 LEFT JOIN ratings rat ON ri.ride_id = rat.ride_id
8 GROUP BY d.driver_id
9 HAVING avg_rating < 3.5 OR avg_rating IS NULL;

```

3.3 Joins for Complex Reports

3.3.1 Multi-Table Trip Report

Listing 3.5: Complete trip report with all details

```

1 SELECT r.ride_id,
2        ru.full_name as rider_name,
3        du.full_name as driver_name,
4        v.vehicle_type, v.plate_num,
5        r.pickup_address, r.dropoff_address,
6        r.fare_amount, r.status
7 FROM rides r
8 INNER JOIN riders ri ON r.rider_id = ri.rider_id
9 INNER JOIN users ru ON ri.user_id = ru.user_id
10 LEFT JOIN drivers d ON r.driver_id = d.driver_id
11 LEFT JOIN users du ON d.user_id = du.user_id
12 LEFT JOIN vehicles v ON r.vehicle_id = v.vehicle_id
13 WHERE r.requested_at >= DATE_SUB(NOW(), INTERVAL 30 DAY);

```

3.4 Views Implementation

3.4.1 Active Rides View

Listing 3.6: ActiveRidesView - Real-time monitoring

```

1 CREATE VIEW ActiveRidesView AS
2 SELECT r.ride_id, r.status,
3        ru.full_name as rider_name, ru.phone as rider_phone,

```

```

4      du.full_name as driver_name, du.phone as driver_phone,
5      v.vehicle_type, v.plate_num,
6      r.pickup_address, r.dropoff_address,
7      r.requested_at, r.fare_amount
8 FROM rides r
9 JOIN riders ri ON r.rider_id = ri.rider_id
10 JOIN users ru ON ri.user_id = ru.user_id
11 LEFT JOIN drivers d ON r.driver_id = d.driver_id
12 LEFT JOIN users du ON d.user_id = du.user_id
13 LEFT JOIN vehicles v ON r.vehicle_id = v.vehicle_id
14 WHERE r.status IN ('Requested', 'Accepted', 'In Progress');

```

3.5 Indexes for Performance

Listing 3.7: Performance optimization indexes

```

1 -- Index on rides for faster rider lookups
2 CREATE INDEX idx_rides_rider_id ON rides(rider_id);
3
4 -- Index for driver ride history
5 CREATE INDEX idx_rides_driver_id ON rides(driver_id);
6
7 -- Index for status-based queries
8 CREATE INDEX idx_rides_status ON rides(status);
9
10 -- Composite index for location-based searches
11 CREATE INDEX idx_locations_city ON locations(city);

```

3.6 Stored Procedures

3.6.1 Fare Calculation Procedure

Listing 3.8: CalculateFare procedure with surge pricing

```

1 DELIMITER //
2 CREATE PROCEDURE CalculateFare(
3     IN p_distance_km DECIMAL(5,2),
4     IN p_duration_min INT,
5     IN p_vehicle_type ENUM('ECONOMY', 'PREMIUM', 'BIKE'),
6     IN p_is_peak BOOLEAN,
7     OUT p_total_fare DECIMAL(10,2),
8     OUT p_base_fare DECIMAL(10,2),
9     OUT p_surge_multiplier DECIMAL(3,2)
10 )
11 BEGIN
12     DECLARE v_base_rate DECIMAL(10,2);
13     DECLARE v_per_km_rate DECIMAL(5,2);
14     DECLARE v_per_min_rate DECIMAL(5,2);

```

```

15
16 -- Get fare rules
17 SELECT base_rate, per_km_rate, per_min_rate
18 INTO v_base_rate, v_per_km_rate, v_per_min_rate
19 FROM fare_rules
20 WHERE vehicle_type = p_vehicle_type;
21
22 -- Calculate base fare
23 SET p_base_fare = v_base_rate +
24                 (v_per_km_rate * p_distance_km) +
25                 (v_per_min_rate * p_duration_min);
26
27 -- Apply surge if peak hour
28 IF p_is_peak THEN
29     SET p_surge_multiplier = 1.5;
30 ELSE
31     SET p_surge_multiplier = 1.0;
32 END IF;
33
34 SET p_total_fare = p_base_fare * p_surge_multiplier;
35 END //
36 DELIMITER ;

```

3.7 Triggers

3.7.1 Payment Status Trigger

Listing 3.9: Auto-update ride status on payment

```

1 DELIMITER //
2 CREATE TRIGGER trg_after_payment_paid
3 AFTER UPDATE ON payments
4 FOR EACH ROW
5 BEGIN
6     IF NEW.payment_status = 'Paid' AND OLD.payment_status != '
7         Paid' THEN
8         UPDATE rides
9         SET status = 'Completed',
10            completed_at = NOW()
11         WHERE ride_id = NEW.ride_id;
12     END IF;
13 END //
14 DELIMITER ;

```

3.8 Events

3.8.1 Promo Code Expiry Event

Listing 3.10: Midnight promo code cleanup

```
1 DELIMITER //  
2 CREATE EVENT evt_expire_promo_codes  
3 ON SCHEDULE EVERY 1 DAY  
4 STARTS CURRENT_DATE + INTERVAL 1 DAY  
5 DO  
6 BEGIN  
7     UPDATE promo_codes  
8     SET is_active = FALSE  
9     WHERE valid_until < NOW()  
10    AND is_active = TRUE;  
11 END //  
12 DELIMITER ;
```

3.9 Data Control Language (DCL)

3.9.1 Role-Based Access Control

Listing 3.11: Creating and granting roles

```
1 -- Create roles  
2 CREATE ROLE admin_role, driver_role, rider_role, support_role;  
3  
4 -- Grant privileges to admin_role  
5 GRANT ALL PRIVILEGES ON rideflow_db.* TO admin_role;  
6  
7 -- Grant limited privileges to driver_role  
8 GRANT SELECT ON rideflow_db.rides TO driver_role;  
9 GRANT SELECT ON rideflow_db.driver_earnings TO driver_role;  
10 GRANT UPDATE (avail_status) ON rideflow_db.drivers TO driver_role  
11 ;  
12  
13 -- Create user and assign role  
14 CREATE USER 'app_driver'@'localhost' IDENTIFIED BY 'password';  
15 GRANT driver_role TO 'app_driver'@'localhost';  
16 FLUSH PRIVILEGES;
```

Chapter 4

Application Development

4.1 Technology Stack

Table 4.1: Technology Stack

Layer	Technology	Purpose
Backend	Node.js + Express.js	Server-side logic
Database	MySQL 8.0	Data storage
Frontend	HTML5 + CSS3 + JavaScript	User interface
Authentication	bcryptjs + express-session	Security
Maps	Google Maps API	Location services

4.2 Backend Architecture

4.2.1 Server.js Structure

Listing 4.1: Express server setup

```
1 const express = require('express');
2 const mysql = require('mysql2/promise');
3 const bcrypt = require('bcryptjs');
4 const session = require('express-session');
5
6 const app = express();
7 const pool = mysql.createPool(dbConfig);
8
9 // Middleware
10 app.use(express.json());
11 app.use(express.static('public'));
12 app.use(session({
13   secret: process.env.SESSION_SECRET,
14   resave: false,
15   saveUninitialized: false
16 }));
```


4.2.2 API Endpoints

Table 4.2: REST API Endpoints

Endpoint	Method	Description
/api/login	POST	User authentication
/api/rider/book	POST	Create new ride booking
/api/driver/toggle-status	POST	Change driver availability
/api/driver/accept-ride	POST	Accept ride request
/api/admin/stats	GET	Dashboard statistics
/api/admin/active-rides	GET	Live ride monitoring

4.3 Frontend Implementation

4.3.1 Responsive Design

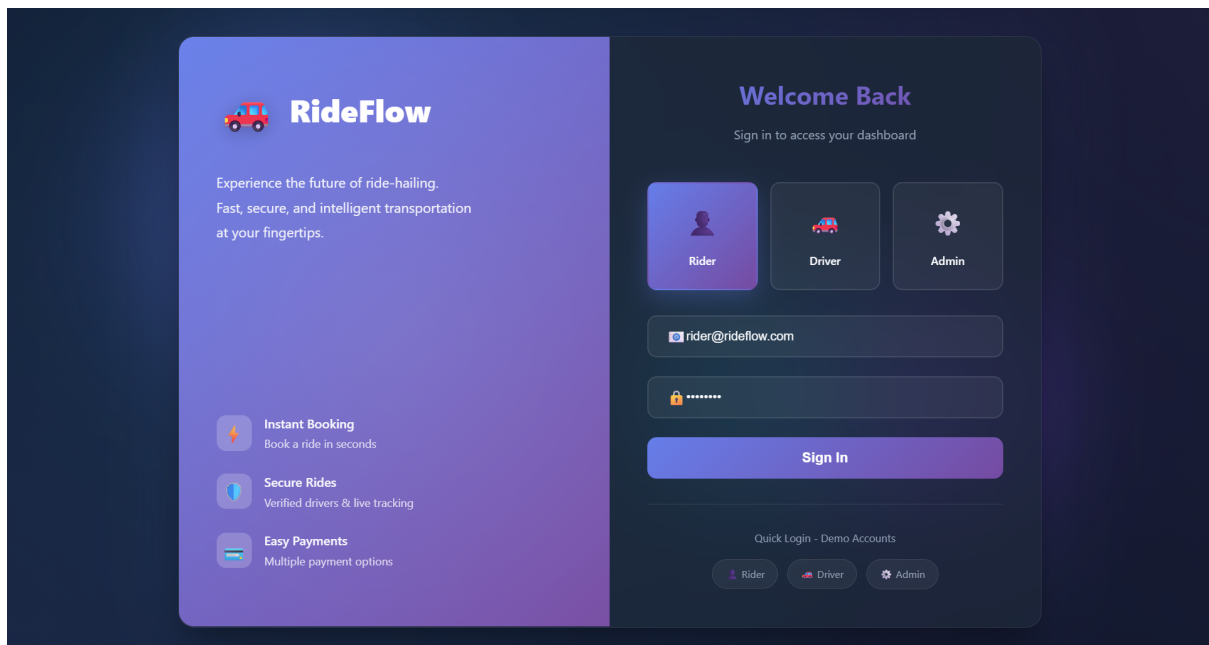


Figure 4.1: Login Page - Entry point for all users

4.3.2 Rider Dashboard

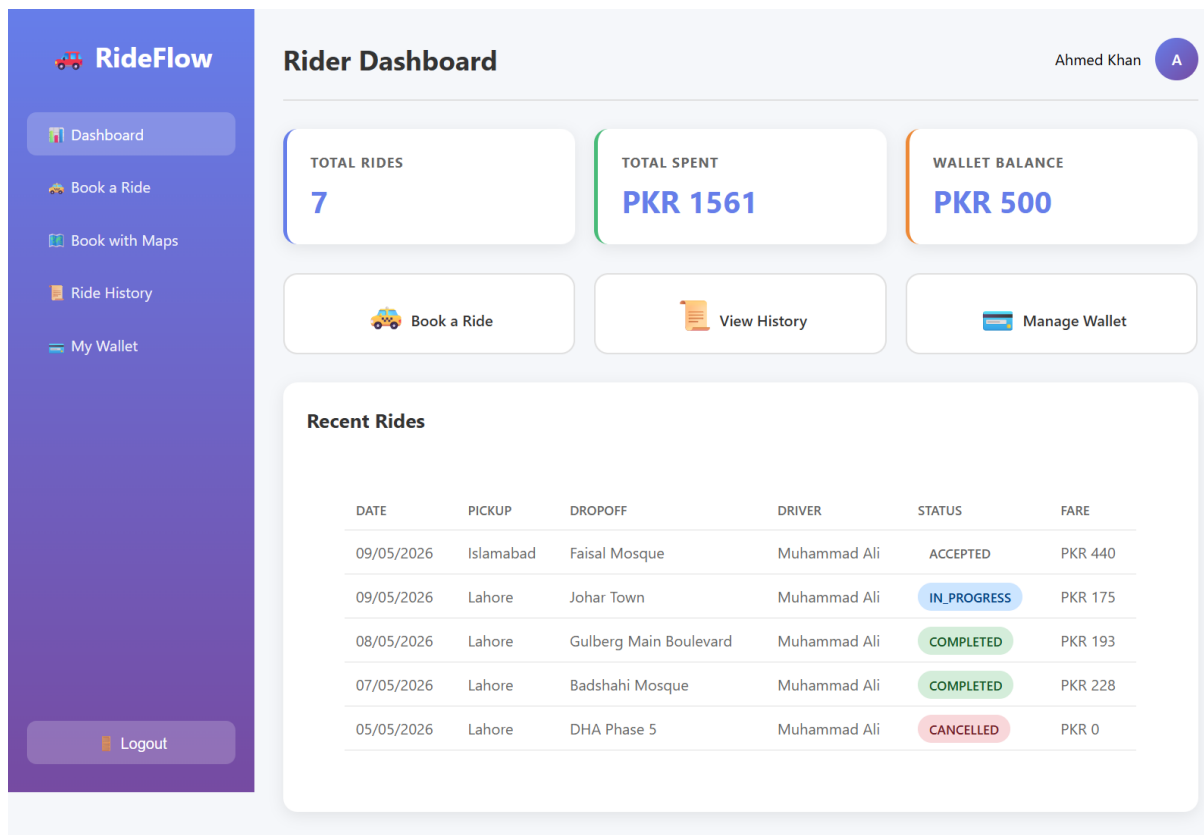


Figure 4.2: Rider Dashboard - Main interface for ride booking

4.3.3 Book with Maps Feature

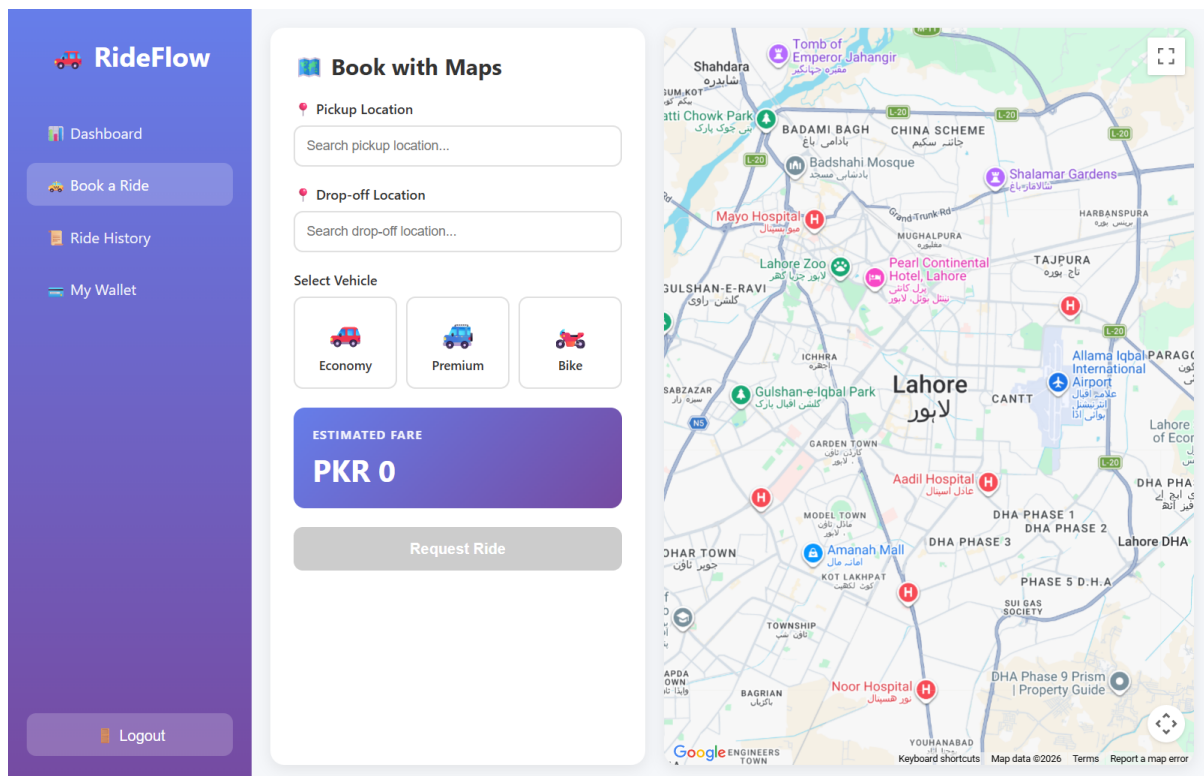


Figure 4.3: Google Maps Integration - Interactive ride booking

4.3.4 Driver Dashboard

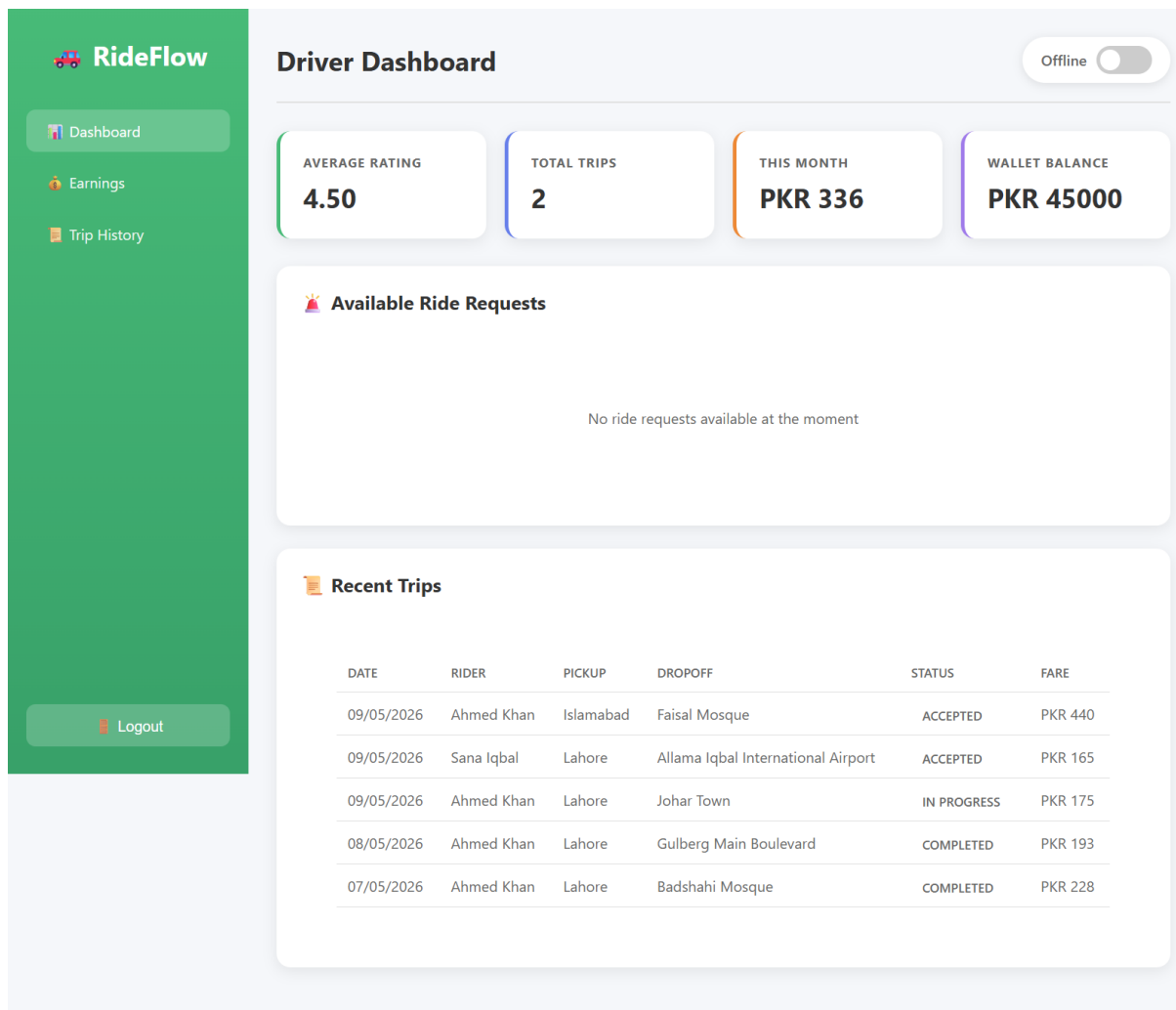


Figure 4.4: Driver Dashboard - Ride acceptance and earnings

4.3.5 Admin Panel

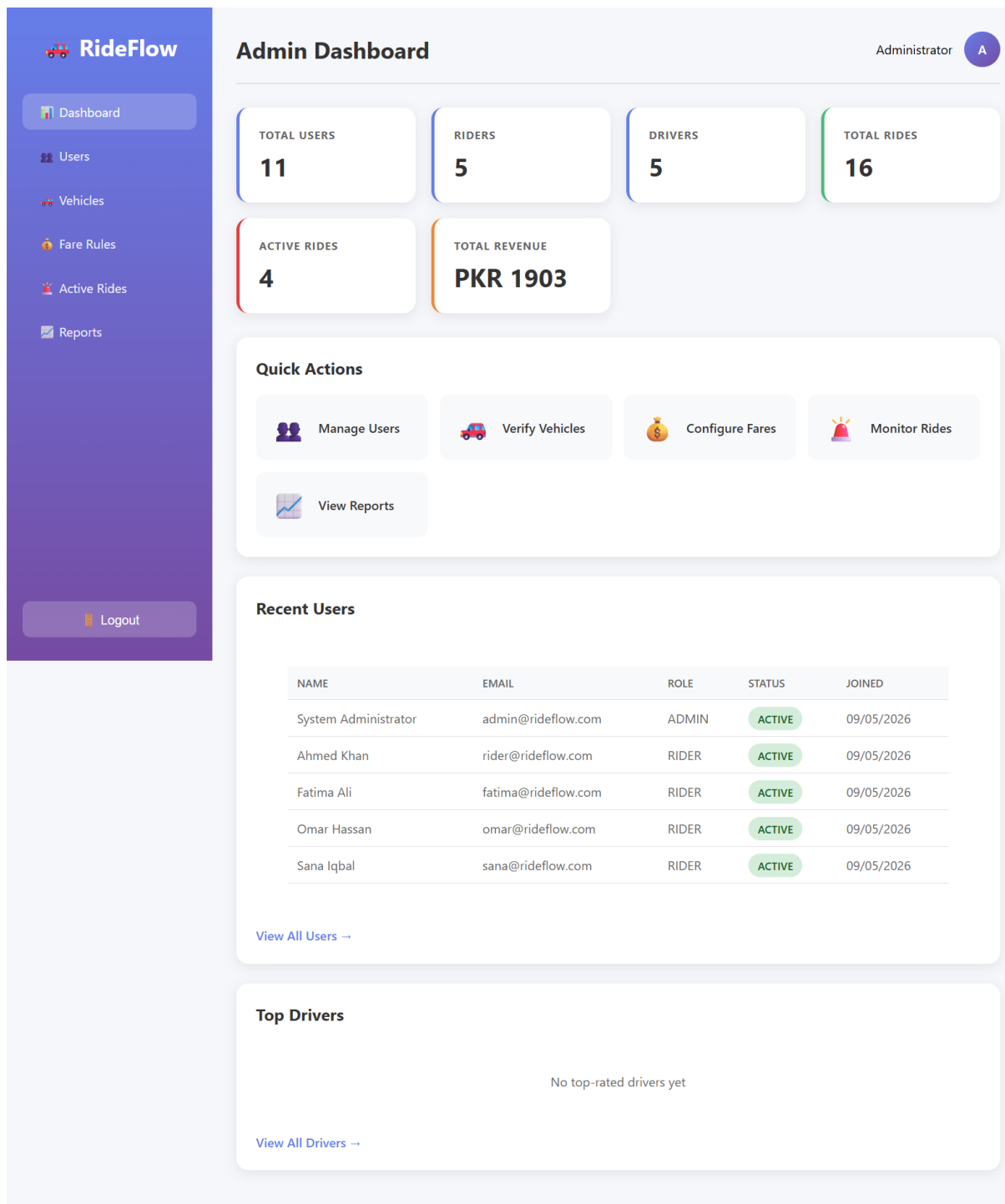


Figure 4.5: Admin Dashboard - System overview and analytics

RideFlow

Dashboard
Users
Vehicles
Fare Rules
Active Rides
Reports
Logout

Manage Users

Search users... All Roles All Status

Riders Drivers All Users

USER ID	NAME	EMAIL	PHONE	ROLE	ACCOUNT STATUS	JOINED	ACTIONS
#2	Ahmed Khan	rider@rideflow.com	03011234567	RIDER	ACTIVE	09/05/2026	Suspend
#3	Fatima Ali	fatima@rideflow.com	03021234567	RIDER	ACTIVE	09/05/2026	Suspend
#4	Omar Hassan	omar@rideflow.com	03031234567	RIDER	ACTIVE	09/05/2026	Suspend
#5	Sana Iqbal	sana@rideflow.com	03041234567	RIDER	ACTIVE	09/05/2026	Suspend
#6	Bilal Ahmad	bilal@rideflow.com	03051234567	RIDER	SUSPENDED	09/05/2026	Activate

Figure 4.6: User Management - Admin control panel

Chapter 5

Features Demonstration

5.1 Core Features

5.1.1 Authentication System

- Role-based login (Rider, Driver, Admin)
- Session management with secure cookies
- Password hashing with bcrypt

5.1.2 Rider Features

1. **Book a Ride:** Simple form-based booking
2. **Book with Maps:** Google Maps integrated booking with:
 - Location autocomplete
 - Visual route display
 - Real-time fare calculation
 - Distance and time estimation
3. **Ride History:** Complete trip records
4. **Wallet Management:** Balance tracking and history

5.1.3 Driver Features

1. **Availability Toggle:** Online/Offline status
2. **Ride Requests:** Accept or reject rides
3. **Earnings Dashboard:** Income tracking
4. **Trip History:** Completed rides log

5.1.4 Admin Features

1. **User Management:** View, filter, suspend users
2. **Vehicle Verification:** Approve/reject driver vehicles
3. **Fare Rules:** Dynamic pricing configuration
4. **Active Rides:** Real-time ride monitoring
5. **Reports:** Revenue analytics and insights

5.2 Advanced Database Features in Action

5.2.1 Stored Procedure: Fare Calculation

The screenshot displays the 'Book a Ride' interface of the RideFlow application. On the left is a purple sidebar with navigation links: Dashboard, Book a Ride (highlighted), Book with Maps, Ride History, My Wallet, and Logout. The main content area is titled 'Book a Ride' and contains the following elements:

- Select Vehicle Type:** Three options are shown: Economy (Affordable rides), Premium (Luxury comfort), and Bike (Quick & nimble). The Premium option is currently selected.
- Pickup Location:** A dropdown menu showing 'Jinnah International Airport, Karachi'.
- Drop-off Location:** A dropdown menu showing 'National Stadium, Karachi'.
- Schedule (Optional):** A date and time input field with a placeholder 'dd/mm/yyyy --:-- --' and a calendar icon.
- ESTIMATED FARE:** A large purple box displaying 'PKR 200'.
- Request Ride:** A blue button at the bottom.

Figure 5.1: Dynamic fare calculation with surge pricing

5.2.2 Trigger: Auto-Complete Ride

When a payment status changes to 'Paid', the trigger automatically updates the ride status to 'Completed'.

5.2.3 Event: Promo Code Cleanup

Every midnight, the event scheduler deactivates expired promotional codes.

5.2.4 View: Active Rides Monitoring

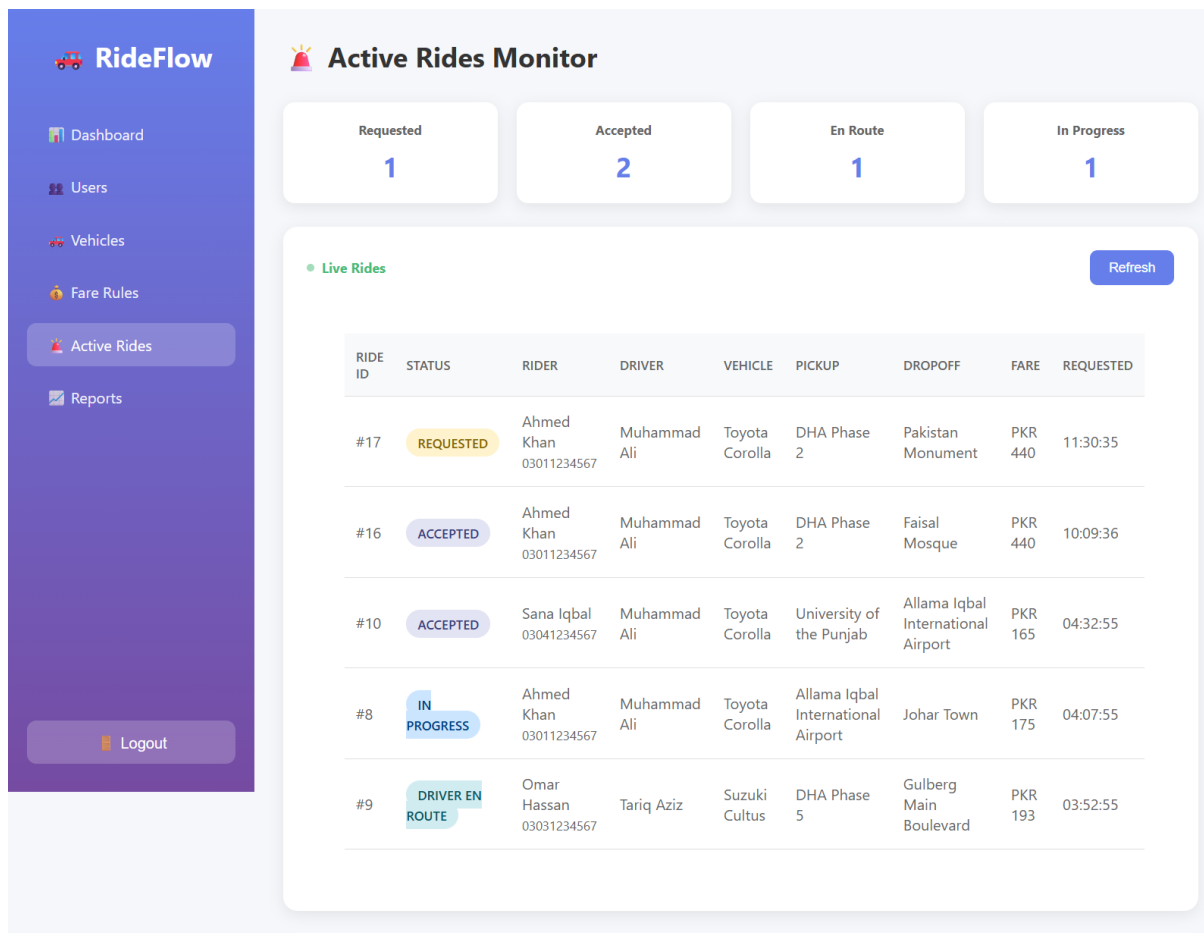


Figure 5.2: Real-time active rides monitoring using ActiveRidesView

Chapter 6

Work Division

6.1 Group Member Responsibilities

Table 6.1: Work Division Between Group Members

Component	Mohid Abbas (24I-0074)	Ubaid ur Rehman (24I-0022)
Database Design	ER Diagram, Schema Design	Relationship mapping, Constraints
SQL Implementation	Views, Stored Procedures, Indexes	Triggers, Events, DCL, Queries
Backend Development	API endpoints, Authentication	Database connection, Session management
Frontend Development	Rider Dashboard, Admin Panel	Driver Dashboard, Login/Signup
Google Maps	Maps API integration	Location services, Geocoding
Testing	Unit testing, Integration testing	SQL testing, UI testing
Documentation	LaTeX Report, Code comments	README files, Setup guides
Deployment	Local deployment testing	Cloud deployment guides

6.2 Contribution Percentage

Figure 6.1: Equal Contribution Distribution

Both group members contributed equally (50% each) to the project.

Chapter 7

Testing and Validation

7.1 Testing Strategy

7.1.1 Database Testing

- SQL query execution validation
- Stored procedure output verification
- Trigger functionality testing
- Event scheduler verification
- Index performance analysis

7.1.2 Application Testing

- Unit testing for API endpoints
- Integration testing for database operations
- UI/UX testing for all user roles
- Security testing for authentication

7.2 Test Cases

Table 7.1: Sample Test Cases

ID	Test Case	Expected Result	Status
TC01	Rider books a ride	Ride created in database	Passed
TC02	Driver accepts ride	Ride status changes to 'Accepted'	Passed
TC03	Payment marked paid	Trigger updates ride to 'Completed'	Passed
TC04	Calculate fare procedure	Correct fare with surge	Passed
TC05	Admin views active rides	ActiveRidesView returns data	Passed

Chapter 8

Deployment

8.1 Local Deployment

8.1.1 Prerequisites

- Node.js (v14 or higher)
- MySQL (v8.0 or higher)
- npm or yarn package manager

8.1.2 Installation Steps

Listing 8.1: Setup commands

```
1 # 1. Database setup
2 mysql -u root -p
3 SOURCE rideflow_relational_schema.sql;
4 SOURCE rideflow_deliverable3_advanced_sql.sql;
5 SOURCE rideflow_seed_data.sql;
6
7 # 2. Application setup
8 cd app
9 npm install
10 copy .env.example .env
11 # Edit .env with your credentials
12
13 # 3. Start server
14 npm start
```

8.2 Cloud Deployment Options

8.2.1 Render.com Deployment

1. Push code to GitHub
2. Connect repository to Render

3. Add environment variables
4. Deploy automatically

8.2.2 Database Hosting

- PlanetScale: Free MySQL hosting
- Aiven: Free 5GB MySQL
- Railway: Simple deployment with database

Chapter 9

Conclusion and Future Work

9.1 Project Summary

RideFlow successfully demonstrates a complete ride-hailing platform implementing all required database concepts. The project includes:

- Comprehensive database design with 10+ tables
- Advanced SQL features (views, procedures, triggers, events)
- Role-based access control
- Functional web application with three user roles
- Google Maps integration for location services

9.2 Challenges Faced

1. Database relationship complexity - Solved through careful ER diagram design
2. SQL syntax differences - Adapted for MySQL 8.0 compatibility
3. Google Maps API integration - Implemented secure key storage
4. Session management - Used express-session with proper configuration

9.3 Future Enhancements

- Real-time driver tracking with WebSockets
- Mobile application using React Native
- Payment gateway integration (Stripe/PayPal)
- AI-based route optimization
- Chat system between riders and drivers
- Push notifications

9.4 Learning Outcomes

- Advanced SQL query writing
- Database design and normalization
- Stored procedures and triggers
- Role-based security implementation
- Full-stack web development
- Third-party API integration

References

1. MySQL 8.0 Documentation - <https://dev.mysql.com/doc/>
2. Express.js Guide - <https://expressjs.com/en/guide/routing.html>
3. Google Maps Platform - <https://developers.google.com/maps>
4. Database Systems: The Complete Book (Garcia-Molina, Ullman, Widom)
5. W3Schools SQL Tutorial - <https://www.w3schools.com/sql/>

Appendix A

Complete File Structure

```
1 RideFlow-DBMS/
2 |-- DELIVERABLE3_README.md
3 |-- DEPLOYMENT_GUIDE.md
4 |-- GOOGLE_MAPS_SETUP.md
5 |-- PROJECT_SUMMARY.md
6 |-- SECURITY_SUMMARY.md
7 |-- README.md
8 |-- rideflow_relational_schema.sql
9 |-- rideflow_deliverable3_advanced_sql.sql
10 |-- rideflow_seed_data.sql
11 |-- erd.drawio
12 |-- erd.drawio.pdf
13 |-- app/
14 |   |-- package.json
15 |   |-- server.js
16 |   |-- .env
17 |   |-- .env.example
18 |   |-- public/
19 |     |-- login.html
20 |     |-- rider/
21 |       |-- dashboard.html
22 |       |-- book.html
23 |       |-- book-with-maps.html
24 |       |-- history.html
25 |       |-- wallet.html
26 |     |-- driver/
27 |       |-- dashboard.html
28 |       |-- earnings.html
29 |       |-- history.html
30 |     |-- admin/
31 |       |-- dashboard.html
32 |       |-- users.html
33 |       |-- vehicles.html
34 |       |-- fare-rules.html
35 |       |-- active-rides.html
36 |       |-- reports.html
```

Appendix B

SQL File Contents Summary

B.1 `rideflow_relational_schema.sql`

Contains CREATE TABLE statements for all database tables including:

- users, riders, drivers tables
- vehicles, rides, locations tables
- payments, ratings, fare_rules tables
- promo_codes, driver_earnings tables

B.2 `rideflow_deliverable3_advanced_sql.sql`

Contains all advanced SQL features organized in sections:

1. Basic Queries (SELECT, WHERE, ORDER BY)
2. Aggregate Functions (SUM, AVG, COUNT, HAVING)
3. Joins for Reports (INNER, LEFT, multi-table)
4. Views (4 views)
5. Indexes (7 indexes)
6. Stored Procedures (5 procedures)
7. Triggers (5 triggers)
8. Events (1 event)
9. DCL (4 roles with GRANT/REVOKE)

Appendix C

Screenshots Index

Table C.1: Required Screenshots for Report

No.	Screenshot Description	File Name
1	FAST University Logo	images/fast_logo.png
2	ER Diagram	images/erd_diagram.png
3	Relational Schema	images/relational_schema.png
4	Login Page	images/login_page.png
5	Rider Dashboard	images/rider_dashboard.png
6	Book with Maps	images/book_with_maps.png
7	Driver Dashboard	images/driver_dashboard.png
8	Admin Dashboard	images/admin_dashboard.png
9	User Management	images/admin_users.png
10	Fare Calculation	images/fare_calculation.png
11	Active Rides	images/active_rides.png

End of Report